Tema 1. Introducción al análisis numérico

Cálculo Numérico

Diego Rojo

Introducción

Análisis Numérico

El **Análisis Numérico** (o Cálculo Numérico) es la rama de la matemática que tiene como objetivo el diseño y estudio de **métodos** que permitan obtener *efectivamente* soluciones numéricas de problemas matemáticos.

Solución numérica

$$\frac{1}{\sqrt{2\pi}} \int_{0.5}^{0.8} e^{-x^2/2} dx$$

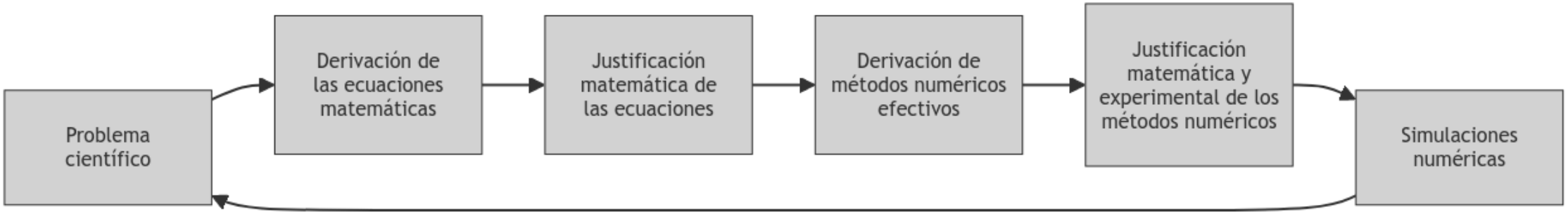
- $P(0.5 \leq X \leq 0.8) \approx 0.096$ con $X \sim N(0,1)$

Efectivamente

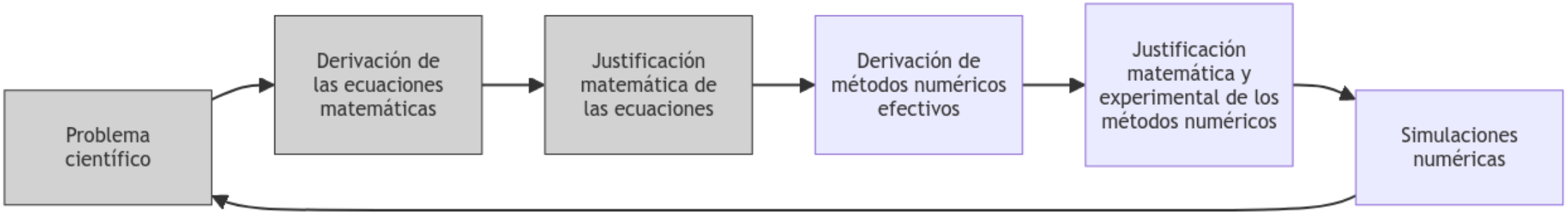
$$A = \begin{bmatrix} a_{1,\,1} & a_{1,\,2} & a_{1,\,3} & \cdots & a_{1,\,25} \\ a_{2,\,1} & a_{2,\,2} & a_{2,\,3} & \cdots & a_{2,\,25} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{25,\,1} & a_{25,\,2} & a_{25,\,3} & \cdots & a_{25,\,25} \end{bmatrix}$$

- ¿Cómo calcular el determinante de A?
- $det(A) = \sum_{\sigma \in S_{25}} \operatorname{sgn}(\sigma) \prod_{i=1}^{25} a_{i,\sigma(i)}$
  - $25! = 15\,511\,210\,043\,330\,985\,984\,000\,000$ términos
  - 1 término por billonésima de segundo
    $15 \cdot 10^{24} \times 10^{-12} = 15 \cdot 10^{12} \approx 475\,000$ años

Proceso


Problema científico
Derivación de las ecuaciones matemáticas
Justificación matemática de las ecuaciones
Derivación de métodos numéricos efectivos
Justificación matemática y experimental de los métodos numéricos
Simulaciones numéricas

Proceso


Problema científico
Derivación de las ecuaciones matemáticas
Justificación matemática de las ecuaciones
Derivación de métodos numéricos efectivos
Justificación matemática y experimental de los métodos numéricos
Simulaciones numéricas

Errores

Errores

- En cálculo numérico, la mayoría de los algoritmos **aproximan** soluciones de problemas matemáticos. La **aproximación** inherentemente introduce **errores**.
- Al realizar los cálculos en un ordenador, una fuente de error inevitable es la **aritmética de precisión finita**.
- En este tema nos centraremos en los errores que surgen de la **representación de números reales** en sistemas informáticos.

Fuentes de error

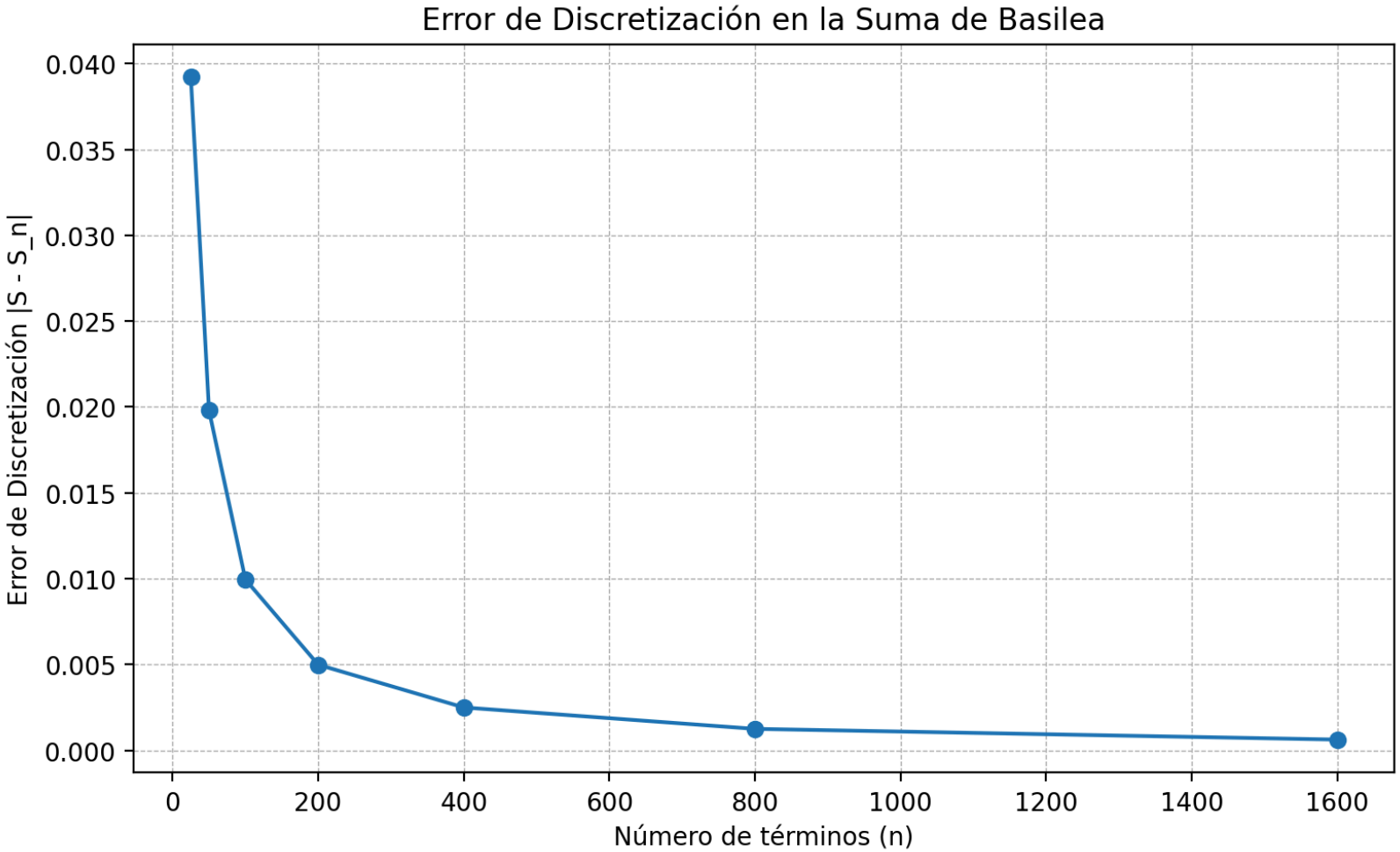
En cálculo numérico, los errores provienen principalmente de dos fuentes:

- **Errores de redondeo:** introducidos por las limitaciones de la aritmética del ordenador al representar y operar con números reales.
- **Errores de discretización (o truncamiento):** ocurren al reemplazar un problema matemático continuo (exacto) por una aproximación discreta que el ordenador puede resolver numéricamente.

Errores de discretización (o truncamiento)

- Ocurren al **aproximar un problema matemático exacto** (continuo) con un problema **discreto** que el ordenador puede resolver.
- **Ejemplo: Problema de Basilea**
  - Suma infinita: $S = 1 + \frac{1}{4} + \frac{1}{9} + \frac{1}{16} + \cdots = \sum_{k=1}^{\infty} \frac{1}{k^2}$
  - Solución analítica (Euler): $S = \frac{\pi^2}{6} \approx 1.644934\ldots$
  - Aproximación numérica (sumar los primeros n términos) $S_n = \sum_{k=1}^{n} \frac{1}{k^2}$
  - **Error de discretización**: La diferencia entre S_n y el valor exacto S.

Ejemplo: Problema de Basilea



Aritmética de precisión finita

- Los ordenadores no pueden almacenar números con infinitos dígitos debido a la **memoria finita**.
  - **Ejemplo**: $\pi \approx 3.14\ldots$ (número irracional con infinitos dígitos).
- Existen muchos números reales que **no pueden representarse** de forma exacta en la aritmética del ordenador.
- En los sistemas numéricos computacionales, **no existe continuidad** como en los números reales. Entre dos números de coma flotante consecutivos, existen **"huecos"**, es decir, números reales que no son representables.

Ejemplo: 1.1 + 0.1

- Un ejemplo ilustrativo de la aritmética de precisión finita es la suma `1.1 + 0.1`.
- Matemáticamente, sabemos que el resultado es **exactamente** 1.2.
- Sin embargo, en Python, al evaluar `1.1 + 0.1 == 1.2`, obtenemos `False`.

```
1.1 + 0.1 == 1.2
```

`False`

- Entonces, ¿cuál es el resultado de la operación?

Ejemplo: 1.1 + 0.1 (cont.)

```
print(1.1 + 0.1)
```

```
1.2000000000000002
```

- El resultado mostrado tiene **16 dígitos** decimales. En sistemas estándar con aritmética de doble precisión, solemos confiar en los primeros **15 dígitos** decimales significativos para números reales de orden O(1).
- Una mayor precisión es posible, pero a costa de **más recursos computacionales**.

Error absoluto y error relativo

- Sea x el **valor exacto** (desconocido en la práctica).
- Sea $\tilde{x}$ un **valor aproximado** (medida, redondeo o resultado algoritmo).
- Definiremos dos formas habituales de cuantificar el error:
  - **Error absoluto**: mide “cuánto” nos desviamos.
  - **Error relativo**: mide “cuánto” nos desviamos **en proporción** al tamaño de x.

Error absoluto

$$\Delta x = \tilde{x} - x$$

Puede ser **positivo o negativo**. Si solo nos importa la magnitud del error:

$$|\Delta x| = |\tilde{x} - x|$$

Cota del error absoluto

A menudo no conocemos Δx, pero sí una **cota**:

$$\left|\Delta x\right| \leq \varepsilon \quad \Longleftrightarrow \quad \left|\tilde{x} - x\right| \leq \varepsilon$$

Entonces el valor exacto x está en el intervalo:

$$\tilde{x} - \varepsilon \leq x \leq \tilde{x} + \varepsilon$$

Error relativo

Si $x \neq 0$, definimos el **error relativo**:

$$\delta x = \frac{\Delta x}{x} = \frac{\tilde{x} - x}{x}$$

Por tanto:

$$\tilde{x} = x(1 + \delta x)$$

- En la práctica, si la aproximación $\tilde{x}$ es suficientemente precisa, podemos usarla para estimar el error relativo:

$$\delta x \approx \frac{\tilde{x} - x}{\tilde{x}}$$

Error relativo vs absoluto

- Relación directa:

$$|\Delta x| = |x| \; |\delta x|$$

- El error relativo suele expresarse en **porcentaje**:

$$100 \cdot |\delta x| \; \%$$

- Ideas clave:
  - **Absoluto** es sensible a la **escala** de x.
  - **Relativo** es más comparable entre magnitudes de distinto orden.

Ejemplo

Considera:

- $x_0 = 1.000,\ \tilde{x}_0 = 0.999$
  - $\Delta x_0 = -10^{-3},\ \delta x_0 = -10^{-3}$
- $x_1 = 10^9,\ \tilde{x}_1 = 10^9 - 1$
  - $\Delta x_1 = -1,\ \delta x_1 = -10^{-9}$
- Aunque $|\Delta x_1| > |\Delta x_0|$, la aproximación de x_1 es muchísimo mejor en términos relativos.

Propagación de errores

- Rara vez usamos un único dato: combinamos aproximaciones en operaciones.
- Pregunta típica: **si a y b tienen error, ¿qué error tendrá $x = f(a, b)$?**

Propagación de errores: Suma y resta

Sea $x = a \pm b$ con aproximaciones $\tilde{a}, \tilde{b}$.

- Resultado clave:

$$|\Delta x| \leq |\Delta a| + |\Delta b|$$

Ejemplo

Si:

$$|4.56 - a| \leq 0.14, \qquad |1.23 - b| \leq 0.03$$

entonces para $x = a - b$:

- $\tilde{x} = 4.56 - 1.23 = 3.33$
- Cota:

$$|3.33 - x| \leq 0.14 + 0.03 = 0.17$$

Cancelación catastrófica

Caso delicado: $x = a - b$ con $a \approx b$.

- El denominador $|a - b|$ puede ser pequeño ⇒ el **error relativo de x se amplifica**.
- Cota:

$$|\delta x| \;\leq\; \frac{|\Delta a| + |\Delta b|}{|a - b|}$$

Esto explica por qué la resta de números “casi iguales” puede ser numéricamente peligrosa.

Ejemplo: Cancelación catastrófica

Si:

$$|1.25 - a| \leq 0.03, \qquad |1.20 - b| \leq 0.03$$

Entonces $\tilde{x} = 1.25 - 1.20 = 0.05$.

Aunque a y b tienen errores relativos del orden del **2–3%**, la cota del error relativo del resultado puede superar el **100%**.

Propagación de errores: Multiplicación y división

Sea $x = ab$ (o $x = a/b$).

- Cota (primer orden):

$$|\delta x| \leq |\delta a| + |\delta b|$$

Ejemplo

Con los mismos datos:

$$|1.25 - a| \leq 0.03, \quad |1.20 - b| \leq 0.03$$

Errores relativos aprox.:

$$|\delta a| \approx \frac{0.03}{1.25} \approx 2.4\%, \qquad |\delta b| \approx \frac{0.03}{1.20} \approx 2.5\%$$

Entonces:

$$|\delta x| \leq 2.4\% + 2.5\% = 4.9\%$$

Propagación de errores: Reglas

- **Suma / resta** → sumar **errores absolutos**:

$$|\Delta(a \pm b)| \leq |\Delta a| + |\Delta b|$$

- **Producto / cociente** → sumar **errores relativos**:

$$|\delta(ab)| \leq |\delta a| + |\delta b|, \qquad |\delta(a/b)| \leq |\delta a| + |\delta b|$$

- **Cancelación catastrófica:** cuidado con $a - b$ si $a \approx b$.

Fenómenos de la aritmética de precisión finita

Cancelación catastrófica

Consideremos la función

$$f(x) = \frac{1 - \cos(x)}{x^2}$$

- $f(x)$ no está definida en $x = 0$.
- ¿Cuál es el límite $\lim_{x \to 0} f(x)$?

Cálculo analítico del límite

Regla de L'Hôpital

$$\lim_{x\to 0} \frac{1-\cos(x)}{x^2} \overset{L'H}{=} \lim_{x\to 0} \frac{\sin(x)}{2x} \overset{L'H}{=} \lim_{x\to 0} \frac{\cos(x)}{2} = \frac{1}{2}$$

Sympy

- Podemos usar `SymPy` para calcular el límite simbólicamente:

```
import sympy

x = sympy.symbols('x')
f_sym = (1 - sympy.cos(x)) / x**2
limit_sym = sympy.limit(f_sym, x, 0)
print(limit_sym) # 1/2
```

Cálculo numérico del límite

- Estimación numérica con Python.
- Evaluamos $f(x)$ para valores de $x = x_i = 10^{-i} \approx 0$, con $i = 5, \ldots, 11$.
- Teóricamente, los valores de $f(x_i)$ deberían aproximarse a 0.5 a medida que x_i se acerca a 0.

```
import numpy as np

f = lambda x: (1 - np.cos(x))/x**2
xi = (1.e-5, 1.e-6, 1.e-7, 1.e-8, 1.e-9, 1.e-10, 1.e-11)
for x in xi:
    print(f'f({x}) = {f(x)}')
```

Cálculo numérico del límite (cont.)

- Inicialmente, la convergencia parece correcta, pero para valores muy pequeños de x, los resultados se vuelven **incorrectos**, tendiendo a 0 en lugar de 0.5.
- Esto se debe a la **cancelación catastrófica**: para valores muy pequeños de x, la diferencia $1 - \cos(x)$ es un número muy cercano a 0.

Solución: Reformulación

- Para mejorar la estabilidad, podemos **reformular** $f(x)$ para evitar la resta de números casi iguales.
- Realizamos transfomaciones algebraicas:

$$f(x) = \frac{1 - \cos(x)}{x^2} = \frac{\sin^2(x)}{x^2(1 + \cos(x))}$$

- Esta nueva fórmula **evita la cancelación catastrófica** para $x \approx 0$.

Falta de asociatividad

- La **falta de asociatividad** en la aritmética de coma flotante significa que el orden en que se realizan las operaciones puede afectar el resultado final.
- En la aritmética real, la suma y la multiplicación son asociativas:
  - $(a + b) + c = a + (b + c)$
  - $(a \times b) \times c = a \times (b \times c)$
- Sin embargo, esto **no siempre** se cumple en la aritmética de coma flotante debido a la precisión finita (errores de redondeo).

Ejemplo: Falta de asociatividad

- Consideremos la suma de tres números con magnitudes muy diferentes:
  - $x = 1.0 \times 10^{20}$
  - $y = -1.0 \times 10^{20}$
  - $z = 1.0$
- En la aritmética *"verdadera"*, $(x + y) + z = x + (y + z) = 1.0$.
- Veamos qué ocurre en la aritmética de coma flotante.

```
x = 1.0e20
y = -1.0e20
z = 1.0

print(f'(x + y) + z = {(x + y) + z}') # 0.0
print(f'x + (y + z) = {x + (y + z)}') # 1.0
```

Precisión de la máquina

- Otro fenómeno inesperado en la aritmética computacional es la **no unicidad del cero** al sumar con uno.
- Sabemos que en matemáticas $0 + 1 = 1$.
- Sin embargo, en un ordenador, existen números **distintos de cero** que, al sumarse a 1, dan como resultado 1 debido a la precisión finita.
- El **épsilon de la máquina**, denotado EPS, es el número positivo más pequeño tal que $1 + \mathrm{EPS} \neq 1$.
- Para cualquier número x entre 0 y EPS (excluyendo 0 y EPS), $(0 < x < \mathrm{EPS})$, se cumple que $1 + x = 1$ en la aritmética del ordenador.

Cálculo del épsilon de la máquina

Podemos aproximar el épsilon de la máquina mediante el siguiente algoritmo iterativo:

1. Inicializar $x = 1.0$.
2. **Mientras** $1.0 + x \neq 1.0$:
    - Dividir $x = x/2.0$.
3. Definir $\mathrm{EPS} = 2.0 \cdot x$.

```
x = 1.0
while (1.0 + x != 1.0):
    x = x / 2.0
EPS = 2.0 * x
print(f'EPS = {EPS}')
print(f'x = {x}')
```

Notación científica normalizada

Notación científica normalizada

- La **notación científica normalizada** es la forma estándar de representar números reales en sistemas informáticos.
- Garantiza que el primer dígito significativo (primer dígito no nulo) esté en una posición específica:
  - **normalizada**: el primer **dígito significativo** está a la **izquierda** del punto decimal.
- Se eligen potencias apropiadas de la base para que el primer dígito significativo se sitúe a la izquierda del punto decimal.
  - Ejemplo: -123.456 se escribe como -1.23456×10^2 (ni -12.3456×10^1 ni -0.123456×10^3).

Notación científica normalizada (cont.)

En general, un número en notación científica normalizada se expresa como:

$$x = \pm r \times b^n$$

Donde:

- $\pm$: **signo** (positivo o negativo).
- b: **base** del sistema (10 decimal, 2 binario).
- r: **mantisa**, con $1 \leq r < b$.
- n: **exponente**, un entero.

Base, mantisa y exponente

- **Base (b)**:
  - La base del sistema numérico.
  - En los ordenadores es binaria ($b = 2$), aunque trabajaremos también con base 10.
- **Mantisa (r)**:
  - Parte de los dígitos significativos del número normalizado.
  - Rango: $1 \leq r < 10$ (base 10) o $1 \leq r < 2$ (base 2).
- **Exponente (n)**:
  - Indica la potencia a la que se eleva la base.
  - Determina la **magnitud** del número.

Aritmética de precisión finita

- Para representar *exactamente* todos los números reales, necesitaríamos una mantisa y un exponente **infinitos**.
- Como esto no es posible, se definen límites:
  - Límites para el exponente n: límite inferior L y límite superior U

$$L \leq n \leq U$$

  - Límites para la mantisa r: número de dígitos significativos t (**precisión**).

Overflow y underflow

- **Overflow (desbordamiento)**:
  - Ocurre cuando el exponente n **excede** el límite superior U.
  - El número es demasiado grande para ser representado.
  - Se representa como `NaN` (*Not a Number*) o `Inf` (*Infinito*).
- **Underflow (subdesbordamiento)**:
  - Ocurre cuando el exponente n es **inferior** al límite inferior L.
  - El número es demasiado pequeño (cercano a 0) para ser representado con precisión.
  - Se aproxima a cero con signo (`+0.0` o `-0.0`).

Ejemplo de desbordamiento (overflow)

- Sistema: $b = 10$, $L = -1$, $U = 1$, $t = 5$
- Números: $x = 12.3$, $y = 34.5$
  - Cálculo: $x \times y = 12.3 \times 34.5 = 424.35$
- En notación científica normalizada: 4.2435×10^2
  - Exponente $2 > U = 1$ **¡Desbordamiento!**
- El número más grande representable en este sistema:

$$9.9999 \times 10^1$$

Ejemplo de subdesbordamiento (underflow)

- Sistema: $b = 10$, $L = -1$, $U = 1$, $t = 5$
- Números: $x = 0.12345$, $y = 0.1$
  - Cálculo: $x \times y = 0.12345 \times 0.1 = 0.012345$
- En notación científica normalizada: 1.2345×10^{-2}
  - Exponente $-2 < L = -1$ **¡Subdesbordamiento!**
- El número más pequeño representable en este sistema:

$$1.0000 \times 10^{-1}$$

Binario (base 2)

- Los sistemas informáticos usan el sistema binario (base 2) para almacenar números en memoria (**bits**).
- Dos formatos comunes para números de coma flotante:
  - Precisión simple (32 bits).
  - Precisión doble (64 bits).

Precisión simple (32 bits)

- Estructura de 32 bits: $b_1 b_2 \ldots b_9 b_{10} \ldots b_{32}$.
- Representa el número real:

$$(-1)^{b_1} \times 2^{(b_2 b_3 \ldots b_9)_2 - 127} \times (1.b_{10} b_{11} \ldots b_{32})_2$$

- 1 bit para el signo, 8 bits para el exponente y 23 bits para la mantisa.
  - Nota: $(1.b_{10} \ldots b_{32})_2$ es la mantisa con el `1.` **implícito** y los bits $b_{10} \ldots b_{32}$ como parte fraccionaria.

Precisión doble (64 bits)

- Estructura de 64 bits: $b_1 b_2 \ldots b_{12} b_{13} \ldots b_{64}$.
- Representa el número real:

$$(-1)^{b_1} \times 2^{(b_2 b_3 \ldots b_{12})_2 - 1023} \times (1.b_{13} b_{14} \ldots b_{64})_2$$

- 1 bit para el signo, 11 bits para el exponente y 52 bits para la mantisa.
  - Nota: $(1.b_{13} \ldots b_{64})_2$ es la mantisa con el `1.` **implícito** y los bits $b_{13} \ldots b_{64}$ como parte fraccionaria.

`floats` en Python

- Python, por defecto, utiliza **precisión doble** (64 bits).
- `numpy.finfo` es una función de `numpy` que proporciona información sobre los límites de los números de coma flotante

`np.finfo(float)`

```
Machine parameters for float64
---------------------------------------------------------------
precision =  15   resolution = 1.0000000000000001e-15
machep =    -52   eps =        2.2204460492503131e-16
negep =     -53   epsneg =     1.1102230246251565e-16
minexp =  -1022   tiny =       2.2250738585072014e-308
maxexp =   1024   max =        1.7976931348623157e+308
nexp =       11   min =        -max
smallest_normal = 2.2250738585072014e-308   smallest_subnormal = 4.9406564584124654e-324
---------------------------------------------------------------
```